



程式設計

Lecture 13

Final project

請做一個自己有興趣的app 例子: <https://huggingface.co/spaces/chenyc0901/edu-cell-counting>

裡面要有plan.md檔，寫下為什麼要做這個app，以及怎麼設計這個app; 另外要使用git 進行檔案追縱; 需部署在免費網站上; 最後加上Demo影片上傳到youtube網站

最後將整個傳到github ; 把github網址貼到excel，網站也貼到excel; youtube Demo也貼到excel

6/15 0:00 前繳交，逾時不候

評分: 創意(5%)，實用度(5%)，美觀(5%)，完成度(80%)，完成難度(5%)



Vibe Coding

用說話的方式寫程式

誰發明的？一句話改變了大家對寫程式的想像

*"Fully give in to the vibes,
embrace exponentials,
and forget that the code even exists."*

— Andrej Karpathy, 2025 年 2 月

Andrej Karpathy 是誰？

他的背景

- OpenAI 共同創辦人之一
- Tesla 前 AI 主管
- Stanford AI 博士
- AI 圈非常有影響力 🧠

這句話是什麼意思？

「順著感覺走，別執著在程式碼本身，
專注在你想要的結果就好。
讓 AI 去煩惱細節吧！」

Vibe Coding 實際上怎麼運作？

1

你說你要什麼

「幫我做一個可以記帳的
網頁，
要能分類支出、看圖表」

2

AI 生成程式碼

AI 直接寫出完整程式碼
(HTML + JavaScript + 樣
式)

3

你看結果對不對

打開網頁看看，哪裡不對
勁
哪裡跟你想的不一樣

4

再說一次調整

「把按鈕改成圓角」
「加一個刪除功能」
反覆直到滿意

核心觀念：你是「導演」，AI 是「演員」，說清楚你要什麼場景就好

Desktop



Codex



Google
antigravity 2.0



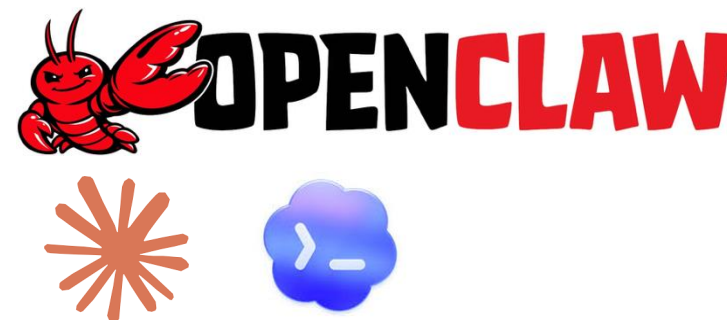
Claude

IDE



Google
antigravity IDE

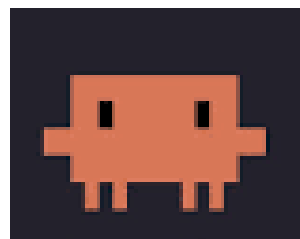
Remote



CLI



Antigravity CLI



Claude code



Codex CLI



Google antigravity IDE

Explorer

Lecture13

.agents

rules

workflows

to_do_list_test

app.js

index.html

style.css

test.md

todo.md

~\$簡報1.pptx

簡報1.pptx

make_slides.py

slides.html

Vibe_Coding_Lecture13.pptx

vibe_coding_teaching_material.json

檔案區

Lecture13



Antigravity IDE

Code with Agent

程式區

Proceeded with

Implementation Plan

畫。

6. 重新整理頁面，確認「學習 antigravity」依然維持完成狀態。

您可以在

成果報告 (walkthrough.md) 中，透過輪播圖表檢視每一個步驟所擷取的高畫質網頁截圖。

工作進度與成果總結：

- 已依據實作計畫在 `to_do_list_test` 目錄下建立原生 HTML、CSS 與 JavaScript 檔案。
- 成功撰寫自動化 Playwright 測試腳本 `test_todo.py`。
- 執行測試並於關鍵節點產出 6 張高畫質 UI 截圖，皆妥善儲存於 `artifacts` 目錄。
- 更新 `task.md` 內所有進度為已完成。
- 建立 `walkthrough.md` 彙整開發內容、測試步驟及截圖輪播。

<> Test Todo

建立自動化測試與截圖腳本 `test_todo.py`。

Walkthrough

這是一份關於極簡美學 Todo List 網頁應用的完整開發與驗證成果報告 (walkthrough.md)。報告詳細說明了...

3 files changed +783 -0 >

Review

AI 區

Review Changes

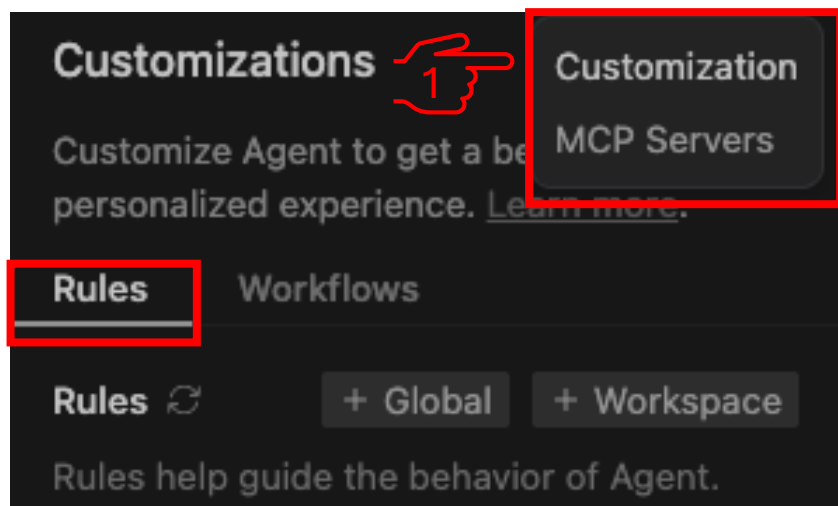
Ask anything, @ to mention, / for actions

Gemini 3.5 Flash (Medium)

Claude: 100% Antigravity - Settings

Rules

是你手動定議的約條件，用來引導Agent遵循特定的行為模式，無論是全域套用到所有專案，還是針對特定工作區設定，Rules都能幫助你打造符合需求的AI助手



- +Global 建立全域規則 (套用到所有工作區)
- +Workspace 建立工作區規則 (只套用到目前的專案)

Rules

是你手動定議的約條件，用來引導Agent遵循特定的行為模式，無論是全域套用到所有專案，還是針對特定工作區設定，Rules都能幫助你打造符合需求的AI助手



+Global 建立全域規則 (套用到所有工作區)

+Workspace 建立工作區規則 (只套用到目前的專案)

儲存在電腦端，會套用到所有工作區。適合定義你個人的編程好和通用規範

個人偏好

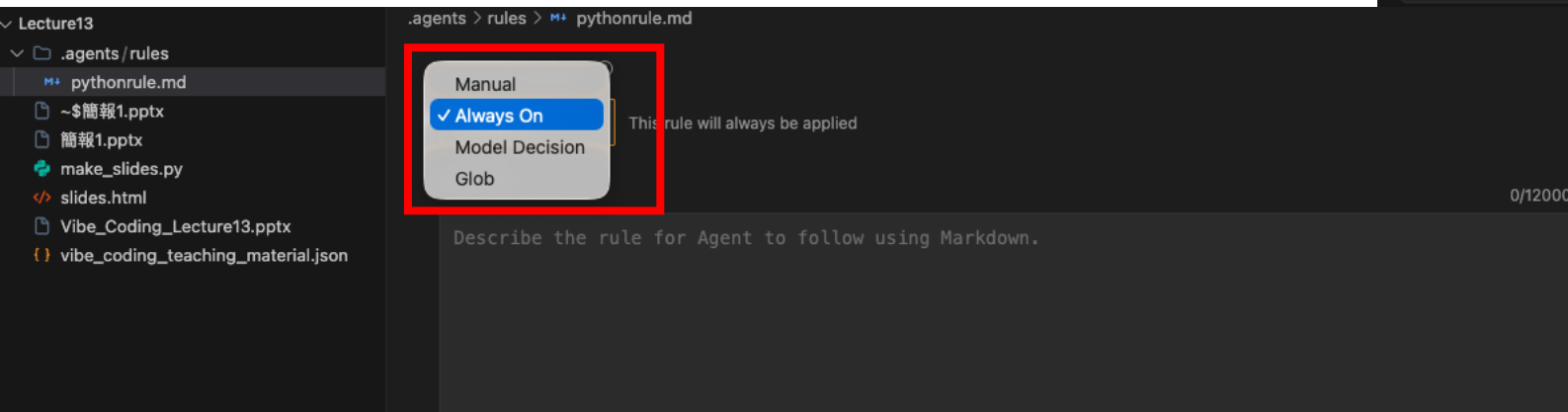
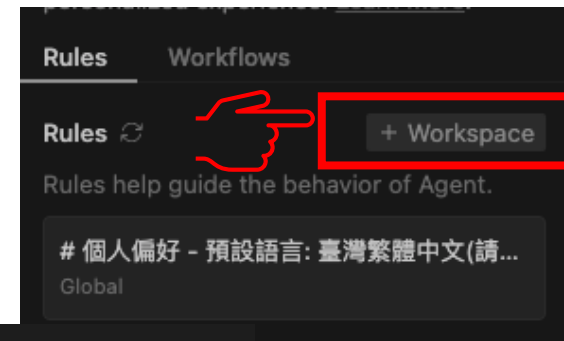
- 預設語言: 臺灣繁體中文(請以繁體中文回應自然語言與文件內容)，包括你自己的思考流程，都使用繁體中文和我交談
- 如果使用python 開發程式，一定要用uv 建立環境，千萬不要用到base及單獨使用pip
- Agent 所產生的implementation plan*, task.md*, walkthrough.md*，全用繁體中文台灣用語
- 如果是javascript或是typescript，一律使用npm/npx，不要亂裝動到我的base環境
- 如果是有其他語言的套件安裝，請用conda管理

Rules

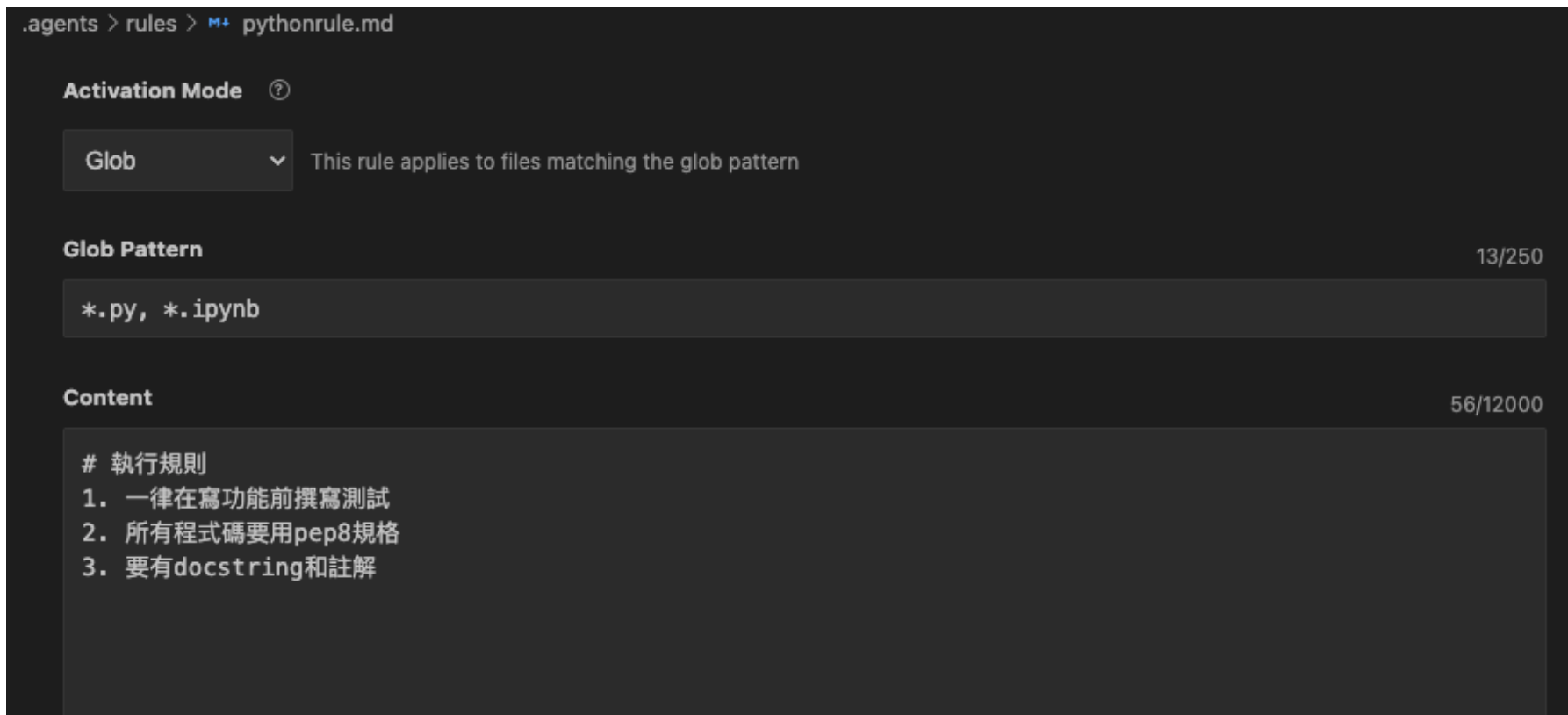
是你手動定議的約條件，用來引導Agent遵循特定的行為模式，無論是全域套用到所有專案，還是針對特定工作區設定，Rules都能幫助你打造符合需求的AI助手

- +Global 建立全域規則 (套用到所有工作區)
- +Workspace 建立工作區規則 (只套用到目前的專案)

儲存在 `.agent/rules/` 資料夾，只套用到該工作區，建立工作區規則時，先輸入規則名稱



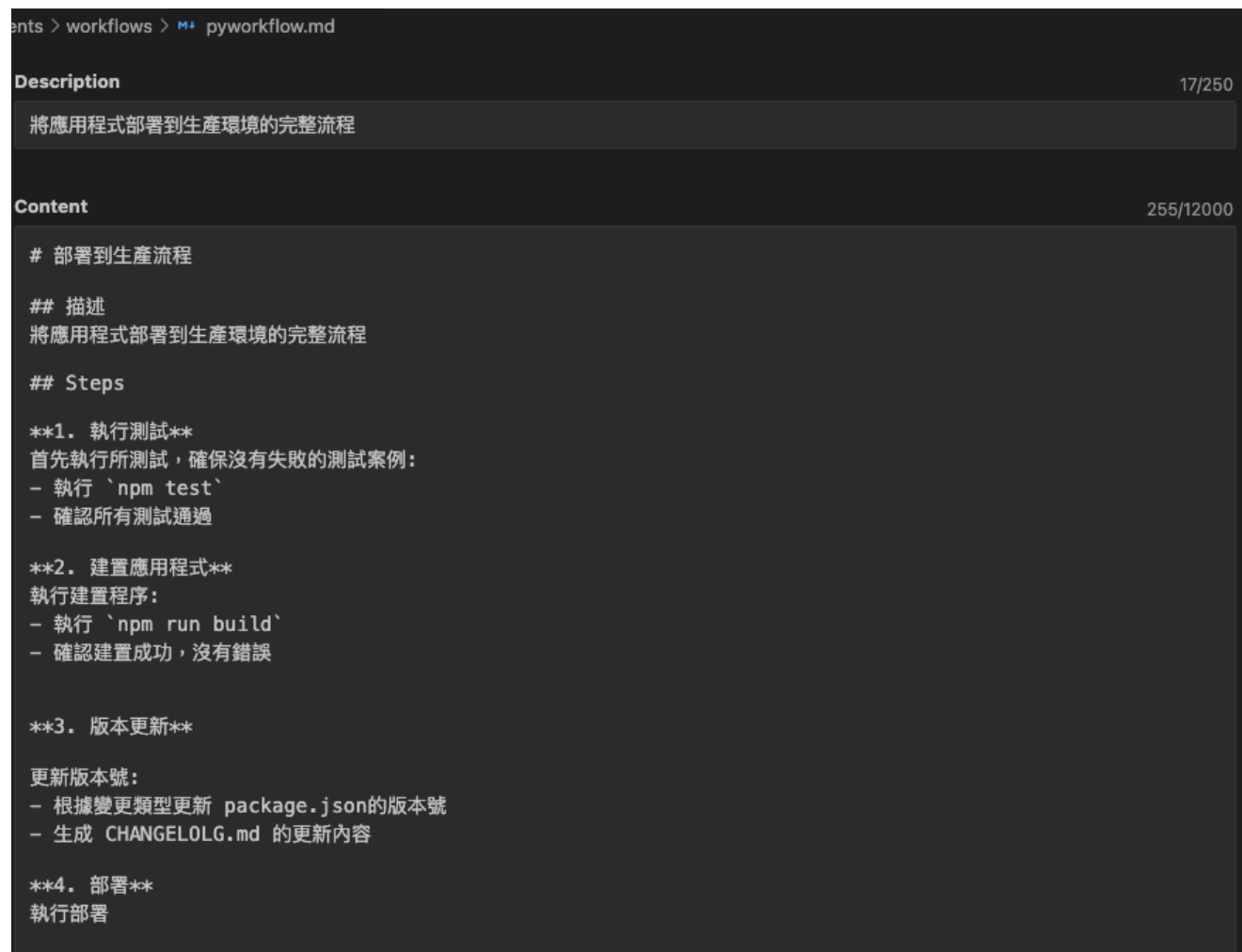
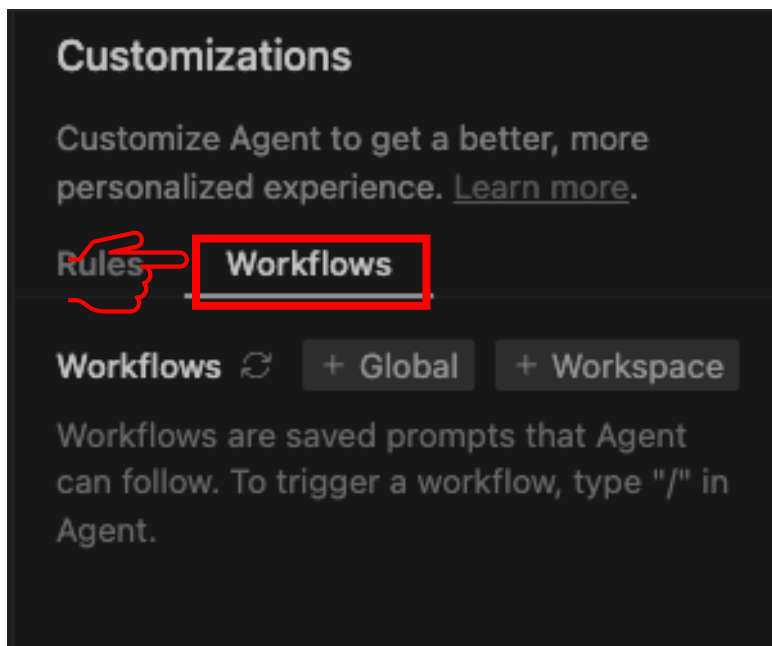
Manual :需要在Agent輸入框中使用@才會套用
Always On :永遠套用
Model decision: 模型自行決定是否套用
Glob :根據glob模式，當符合的檔案被編輯時自動套用



設定Glob Pattern為 `*.py *.ipynb`，當編輯Python或Jupyter檔案時會自動套用

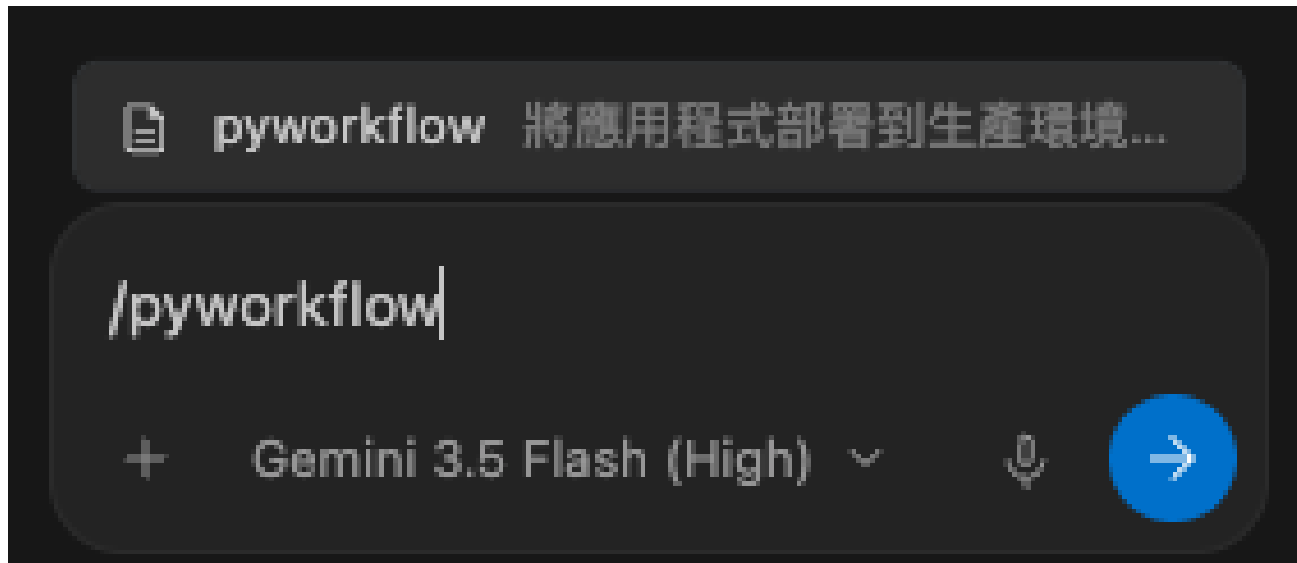
Workflows: 定義自動化工作流程

讓你能夠定義一系列的步驟，引導Agent完成重複性的任務序列。Workflows → 步驟導向，Rules → 約束導向



Workflows: 定義自動化工作流程

執行剛定義好的workflow



M+ todo.md ×



to_do_list_test > M+ todo.md

```
1 請幫我建立一個Todo List網頁應用在這個資料夾，需求如下：
2 1. 可以新增待辦事項(輸入文字按下Enter 或點擊按鈕皆可新增)
3 2. 可以標記事項為完成/未完成(點擊切換)
4 3. 可以刪除待辦事項
5 4. 可以篩選顯示：全部、未完成、已完成
6 5. 使用localStorage 儲存資料，重新整理不會遺失
7
8 技術要求
9 1. 不要使用任何框架或函式庫，使用原生HTML、CSS、JavaScript即可
10 2. CSS使用現代化設計，支援響應式佈局
11 3. JavaScript 使用ES6+語法
12 4. 程式碼架構清晰，加入適當註解
13 %L for Agent
```

Artifacts

任務開始前

agent 先產生 task list 或 implementation plan，讓你看它是否理解需求。

實作前

agent 提出具體變更計畫，可能要求你批准。

實作中

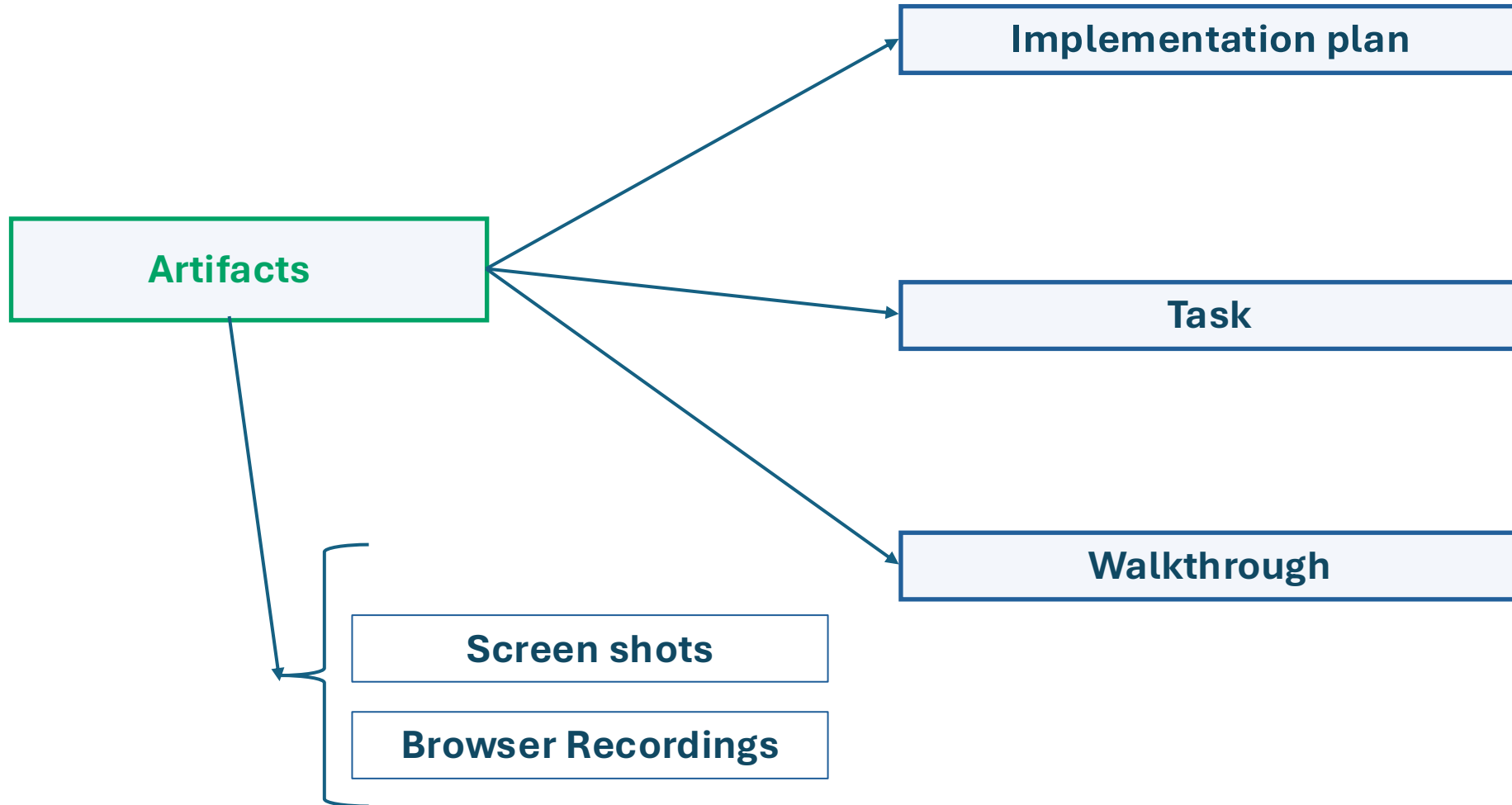
agent 更新任務清單、產生 diff、記錄變更。

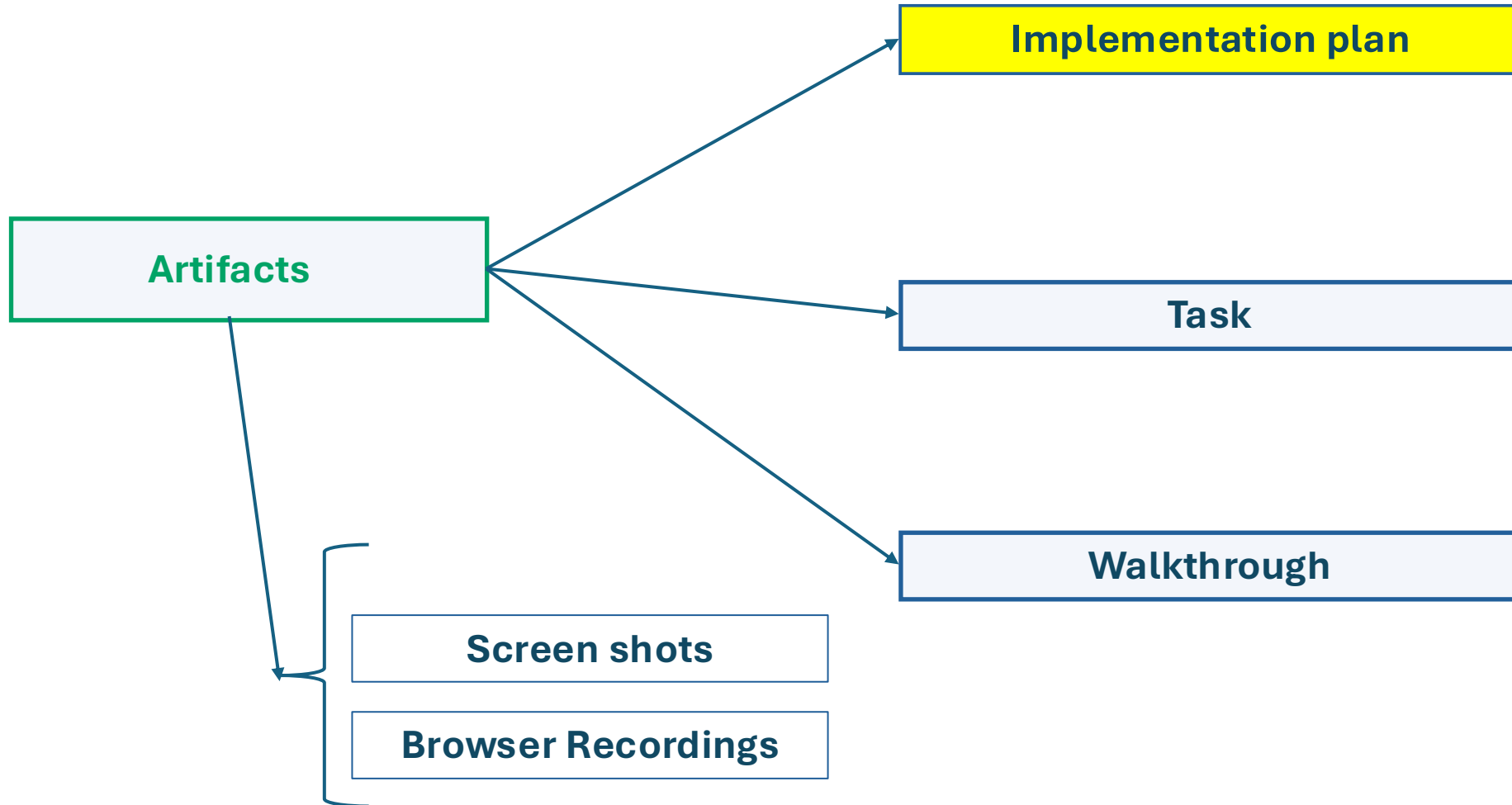
驗證時

agent 產生測試結果、browser screenshot、browser recording。

完成後

agent 產生 walkthrough，總結完成內容與驗證方式。





Implementation Plan：實作計畫

Implementation Plan 是最重要的 artifact 之一。它會說明 agent 打算怎麼改你的 codebase。

它通常會包含：

要改哪些檔案

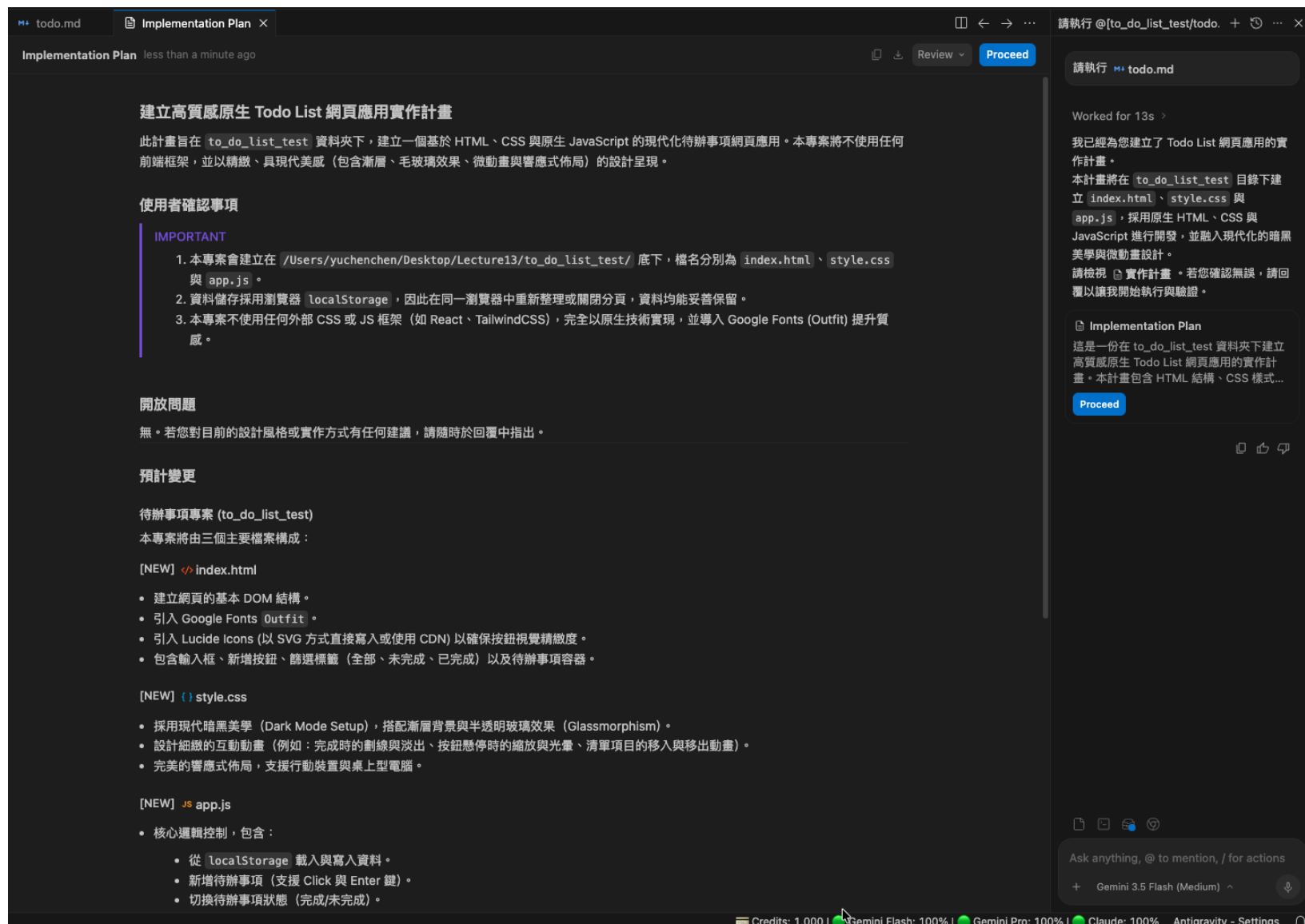
為什麼要改這些檔案

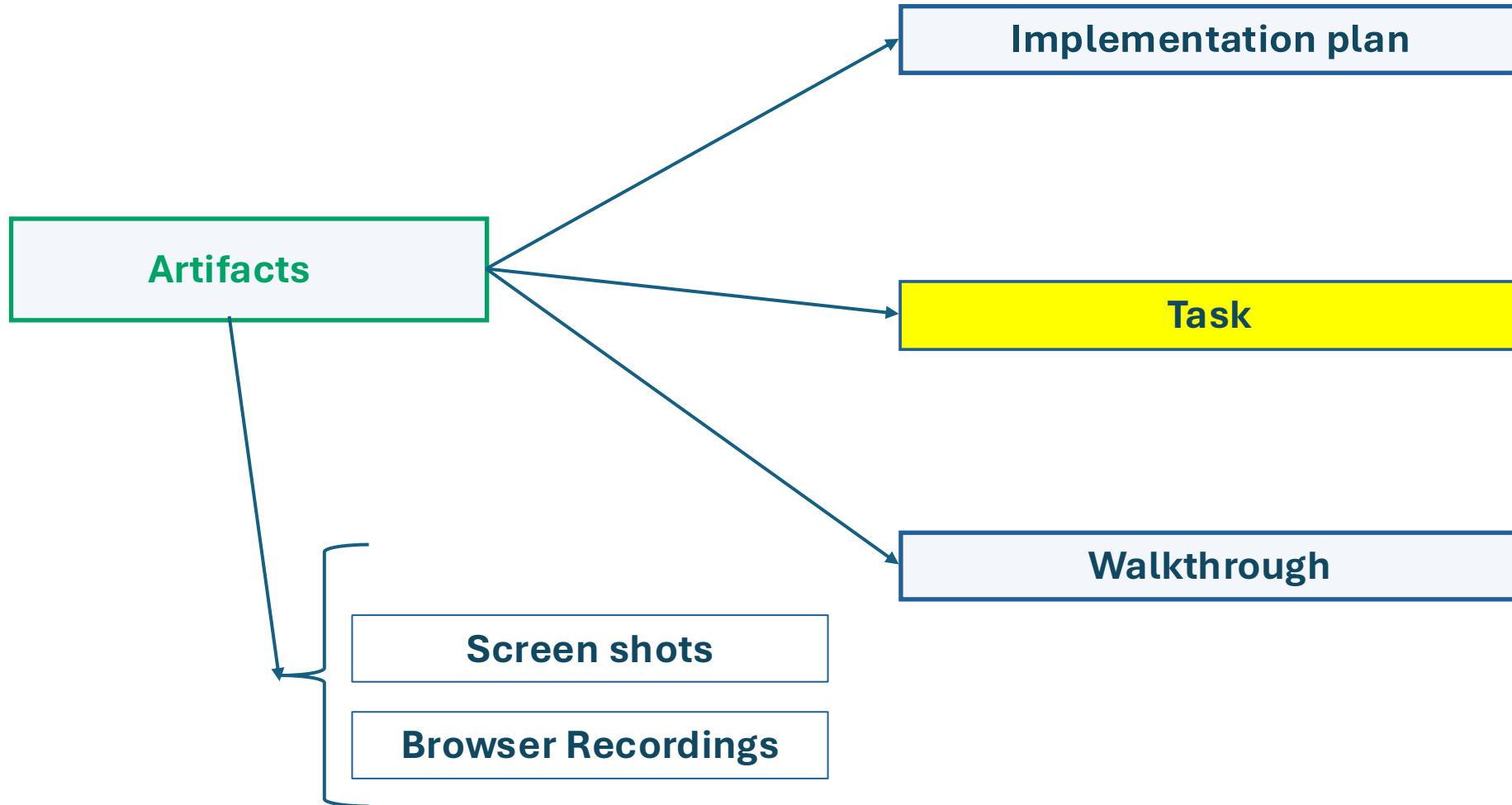
預計新增或刪除哪些邏輯

資料流或 API flow 如何變化

風險與測試方式

可能的替代方案

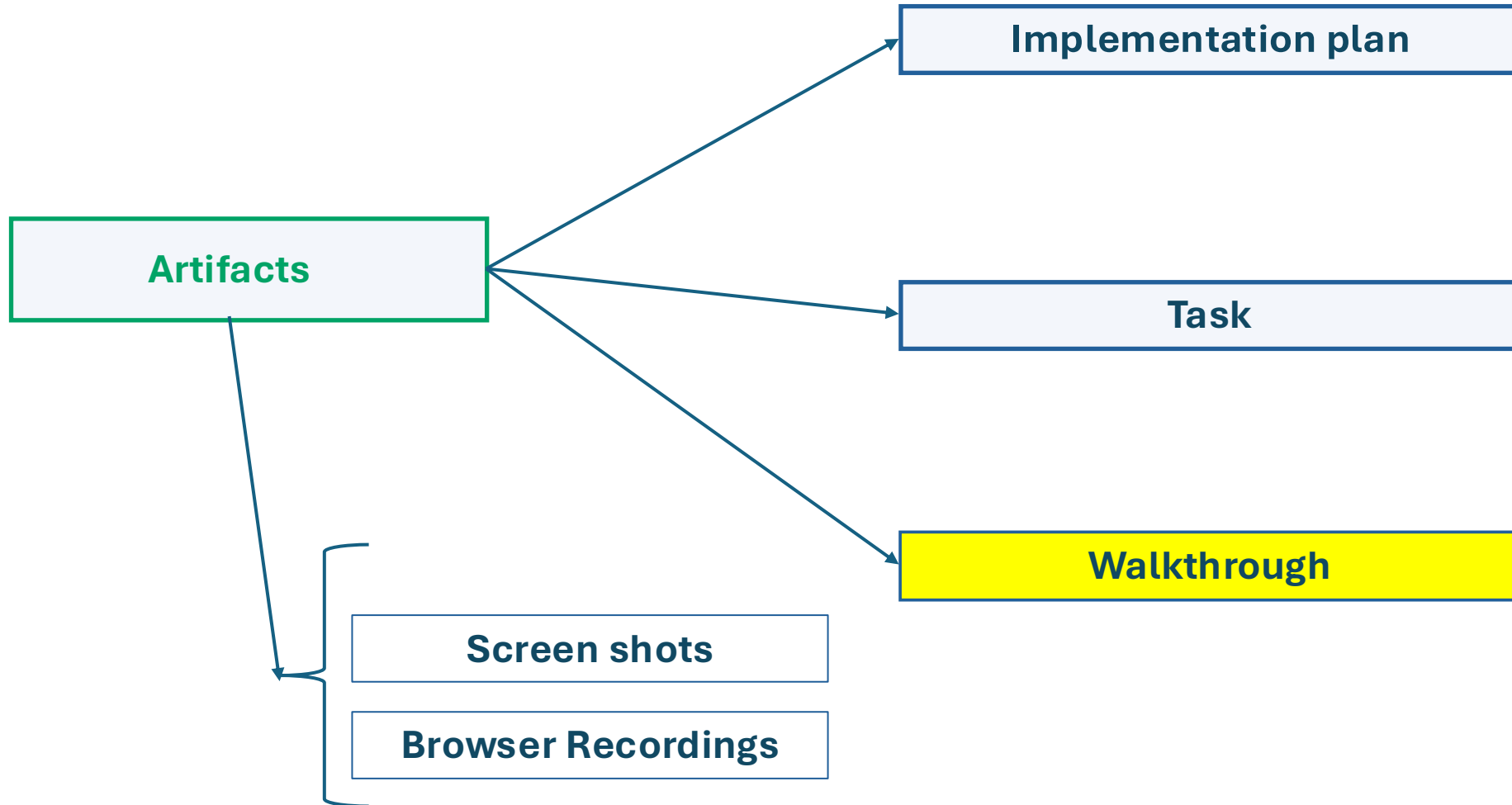




Task List：任務清單

Task List 是 agent 在開始工作前或工作中產生的結構化待辦清單。
Google 文件稱 task list 是 agent 用來組織工作、驗證等流程的 artifact。

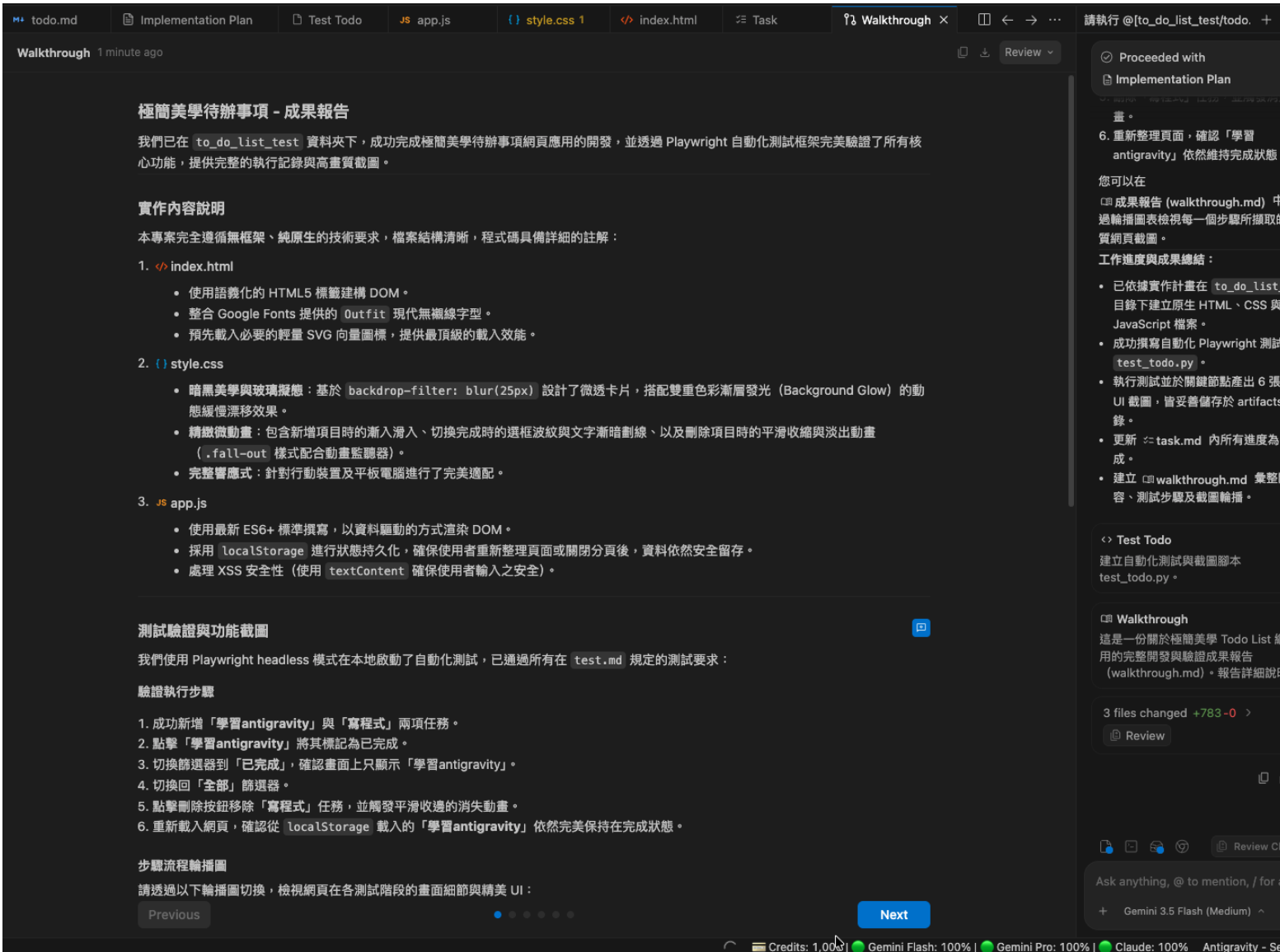


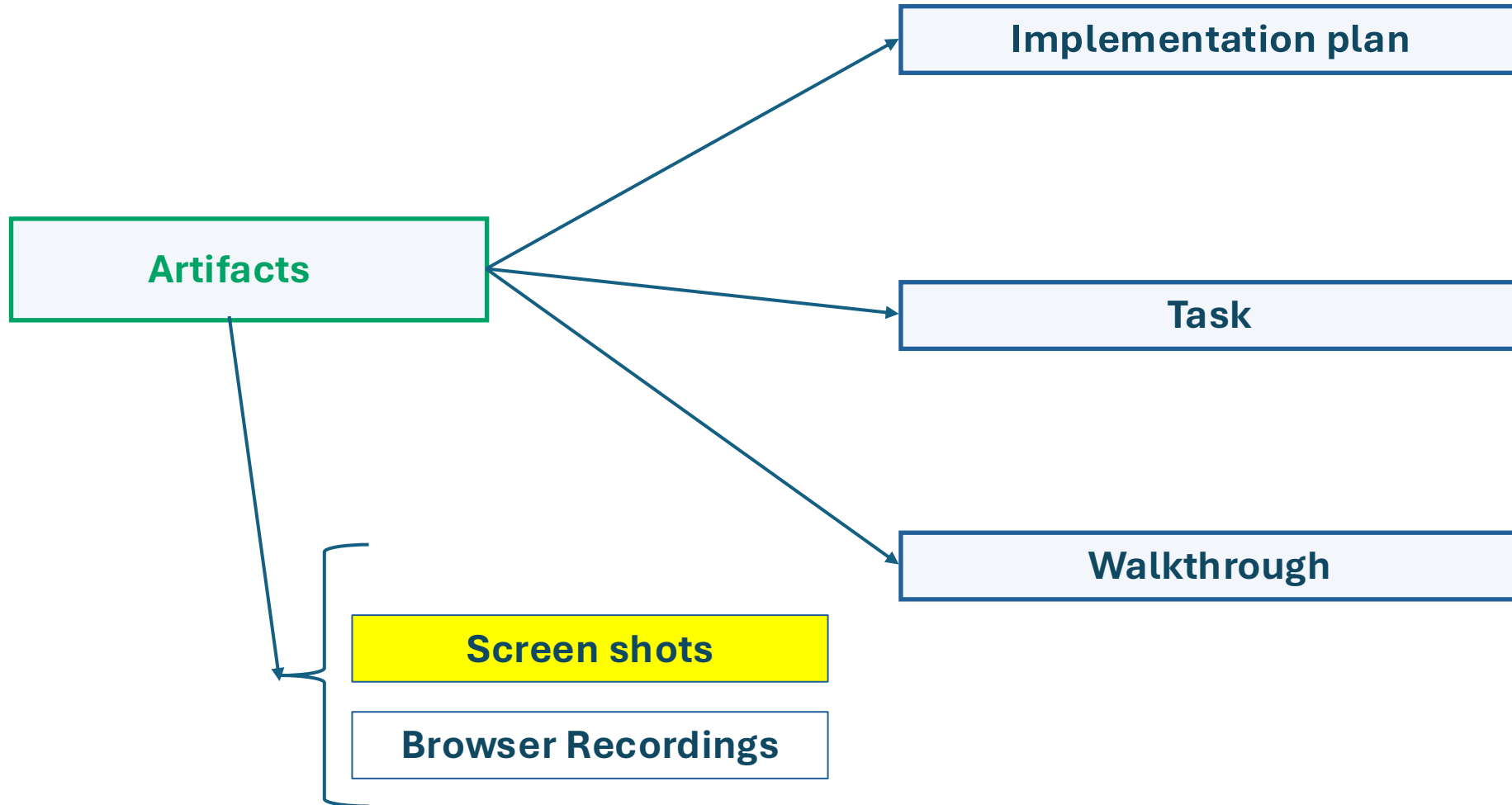


Walkthrough：工作導覽／交付說明

Walkthrough 可以理解成 agent 完成任務後的「交付報告」。

它通常會總結：
做了哪些修改
哪些檔案被改為什麼這樣改
怎麼驗證
還有哪些限制或後續建議
這很像一個 junior developer 做完任務後寫給 reviewer 的說明





Screenshots：截圖

如果任務涉及 UI、網站、瀏覽器操作，**agent** 可能會產生 **screenshots**。

官方文件指出，**screenshots** 會被存成 **image artifacts**，而且可以被評論，用來給 **agent** 回饋。

例如你要求：

幫我調整 **checkout page** 的 **layout**。

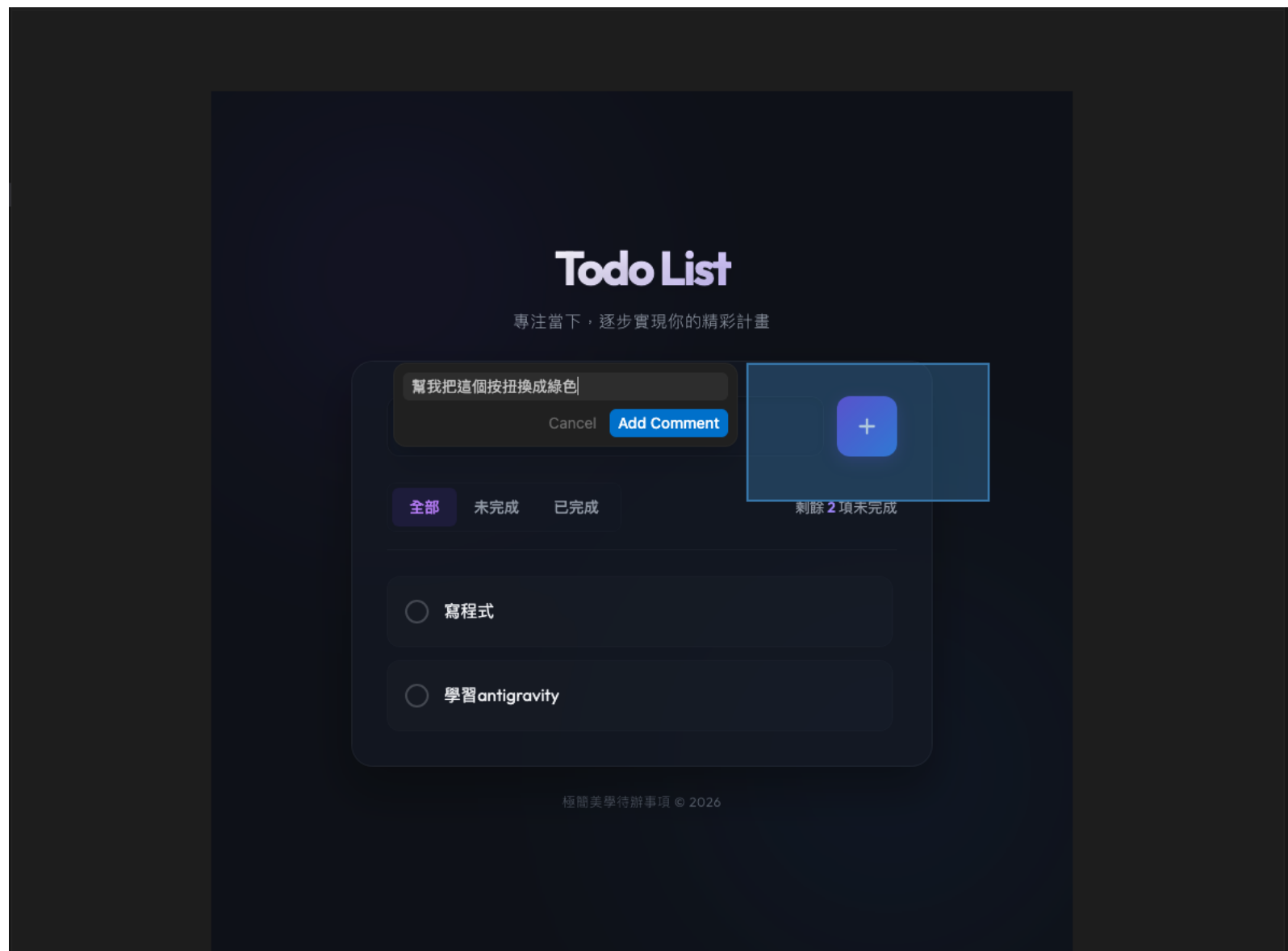
agent 完成後可能會提供一張瀏覽器截圖。你可以直接在截圖 **artifact** 上指出：

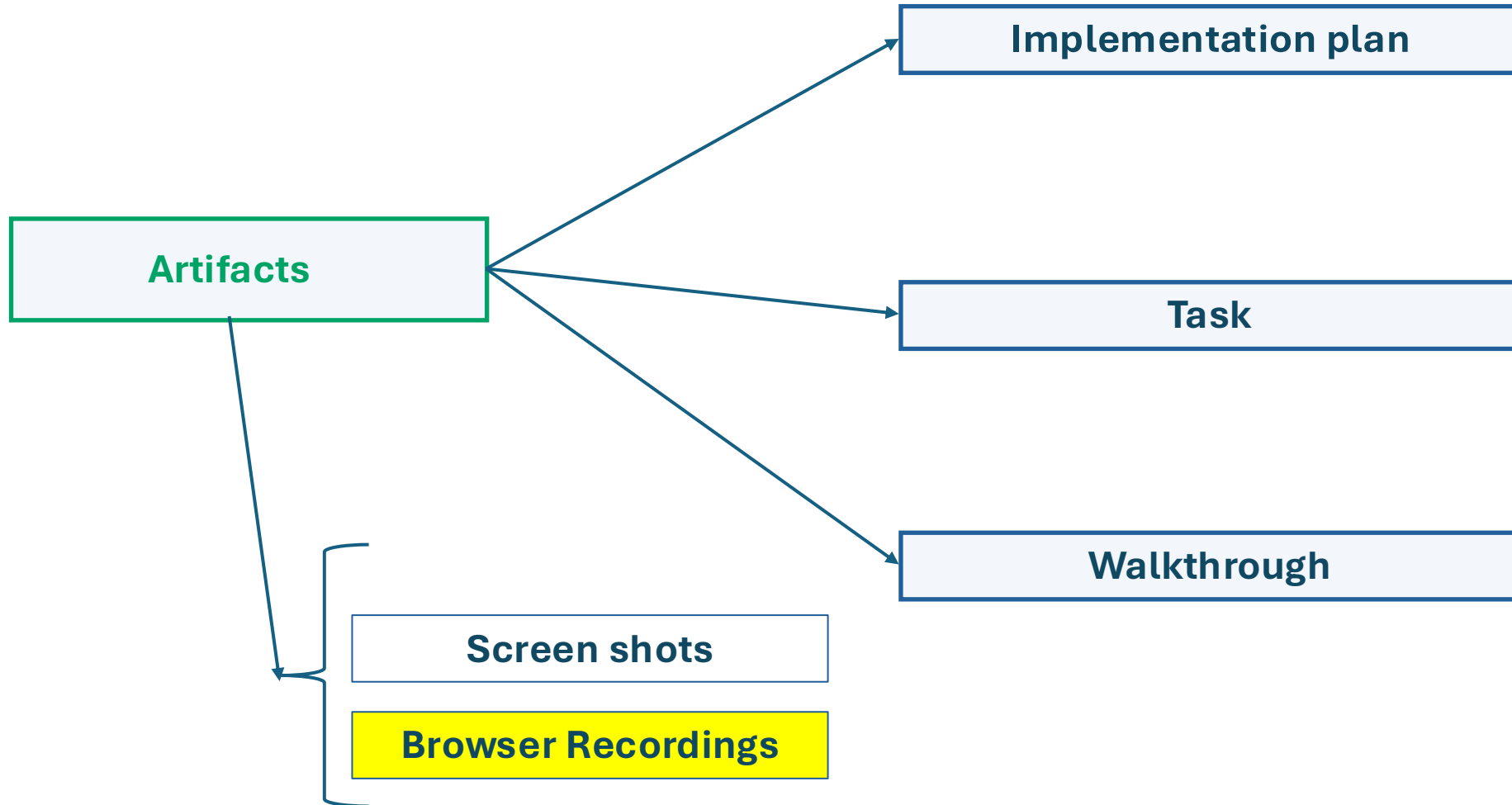
「這個按鈕太靠右」

「這裡文字被截斷」

「手機版 **navbar** 壞掉了」

「這個 **modal** 的 **z-index** 不對」





Browser Recordings

Browser Recordings 是更進階的驗證 artifact 。

Antigravity 的瀏覽器子 agent 在操作 browser 時，可能會產生 recording；官方文件說每次 browser subagent 操作瀏覽器時，它可能會生成錄影



SKILL

讓 AI 學會你想要的「做事方式」

什麼是 SKILL？

SKILL 就像是一本「SOP 手冊」，告訴 AI 碰到某種狀況要怎麼做

寫好一個 SKILL 之後，你只要打 /skill 名稱，或直接說出來，AI 就知道要照那個流程做



像「食譜」

你給 AI 一份食譜（Skill），
它就知道做這道菜要按什麼步驟



AI 自動判斷

你說「幫我看看有啥改動」
AI 自己知道要用
「整理 git 差異」的 Skill



或你主動叫

你直接打 /code-review
AI 就用審查程式碼的方式來幫你

SKILL 的作用 → 有點像class的味道，重覆再利用

每個skills 應該只做一件事，而且要做好，不要建立萬能的skill，而是為不同的任務建立獨立的skills

/notebooklm

建立 Google NotebookLM 筆記本，幫你整理資料或生成 Podcast

/code-review

審查你的程式碼，找出問題、給改善建議

/vercel:deploy

把你的網站部署上線（一個指令搞定）

/security-review

檢查你的程式有沒有安全漏洞

/loop

讓某件事每隔一段時間重複執行

💡 你可以寫自己的 SKILL！只要用一個 Markdown 文件說明「遇到 OO 情況要怎麼做」就完成了

怎麼叫 AI 用 SKILL？兩種方式



你主動叫它

直接打斜線加指令名稱

/code-review

/deploy

/security-review

/summarize-changes

適合：你清楚知道要做什麼



AI 自動判斷

你說的話讓 AI 自己判斷要用哪個 Skill

你說：「這次有哪些改動？」

→ AI 自動用整理 git 差異的 Skill

你說：「幫我看看 PR 有沒有問題」

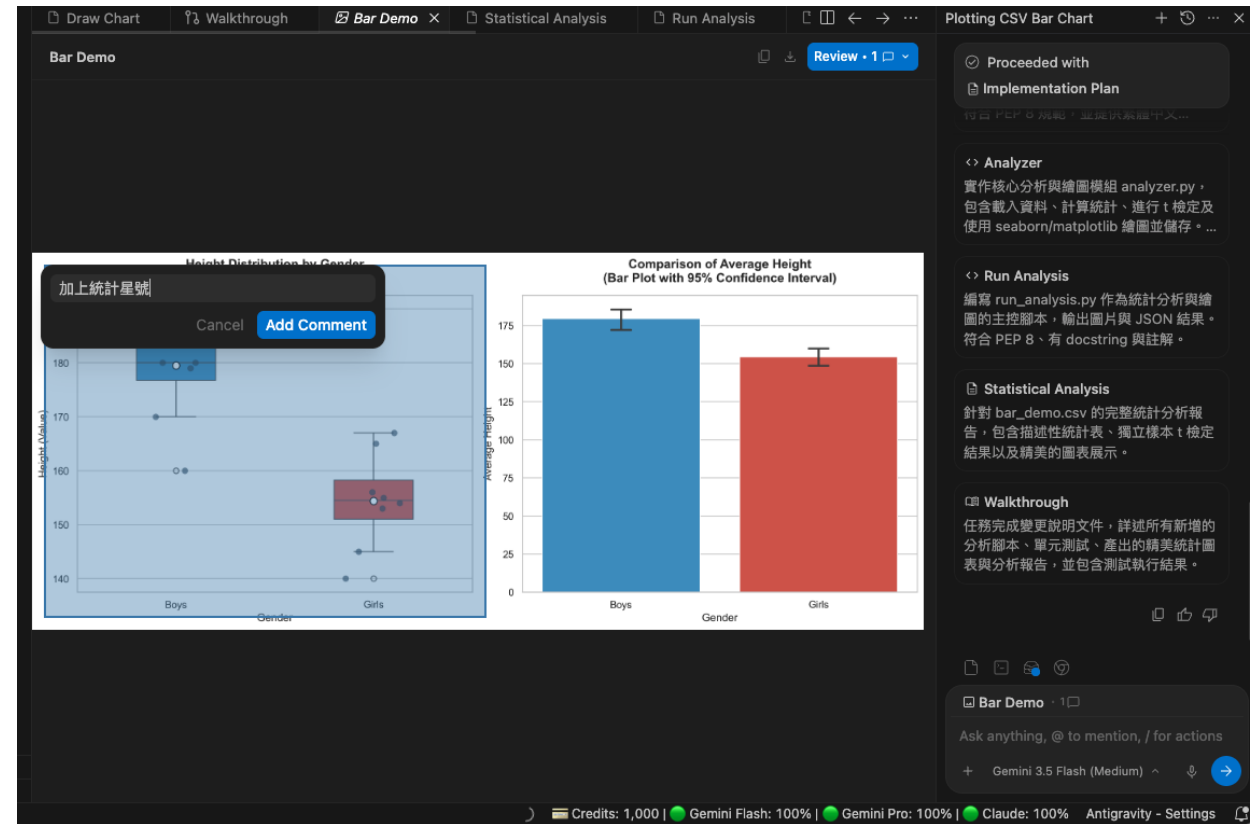
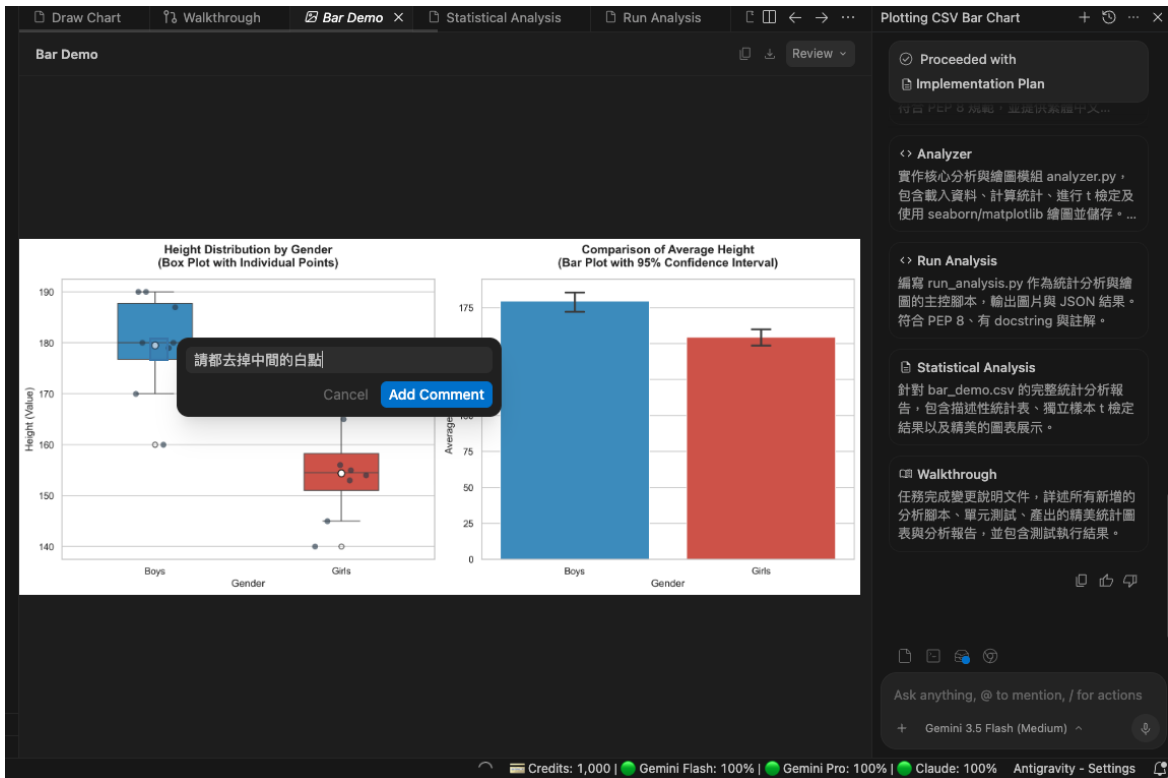
→ AI 自動用程式碼審查 Skill

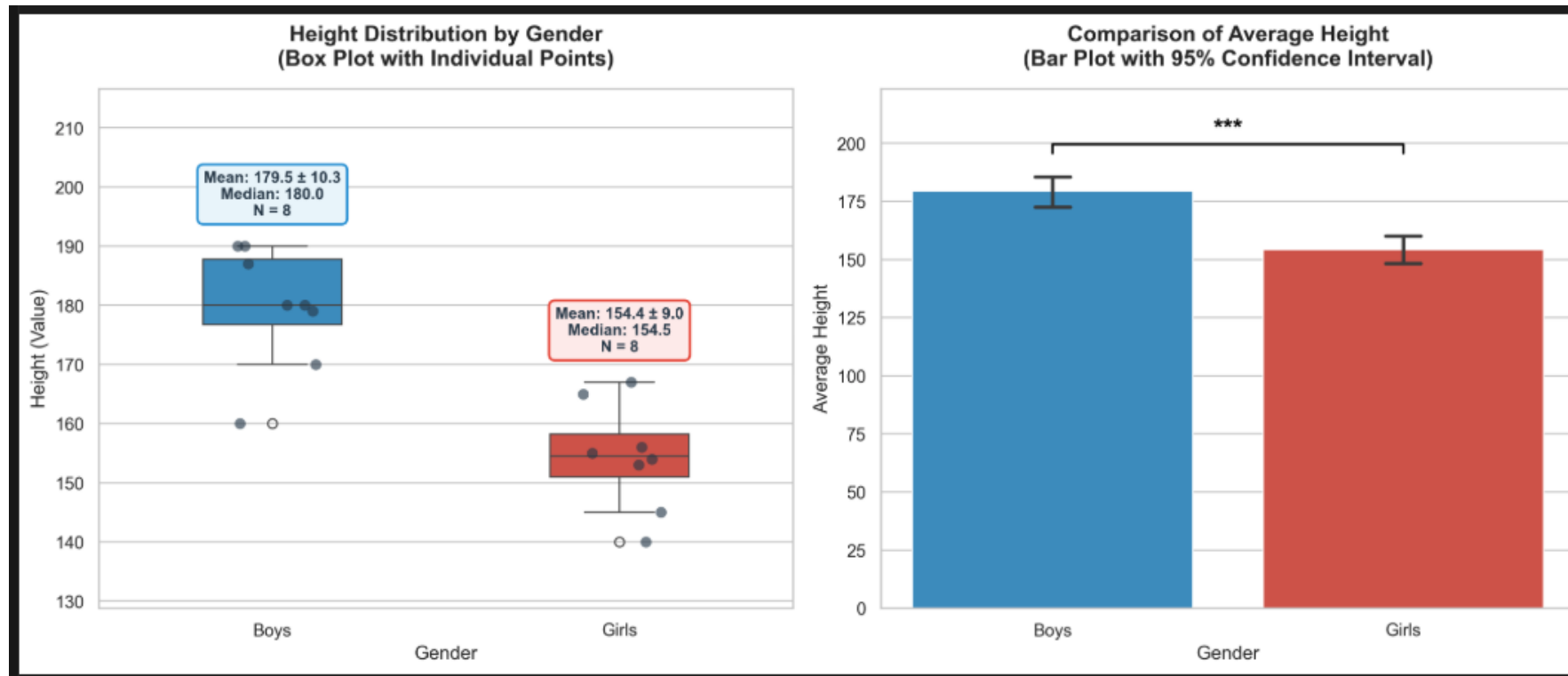
你說：「幫我建一個筆記本整理資料」

→ AI 自動用 notebooklm Skill

適合：你用自然語言描述問題

@bar_demo.csv 請根據裡面的內容幫我
繪製統計圖

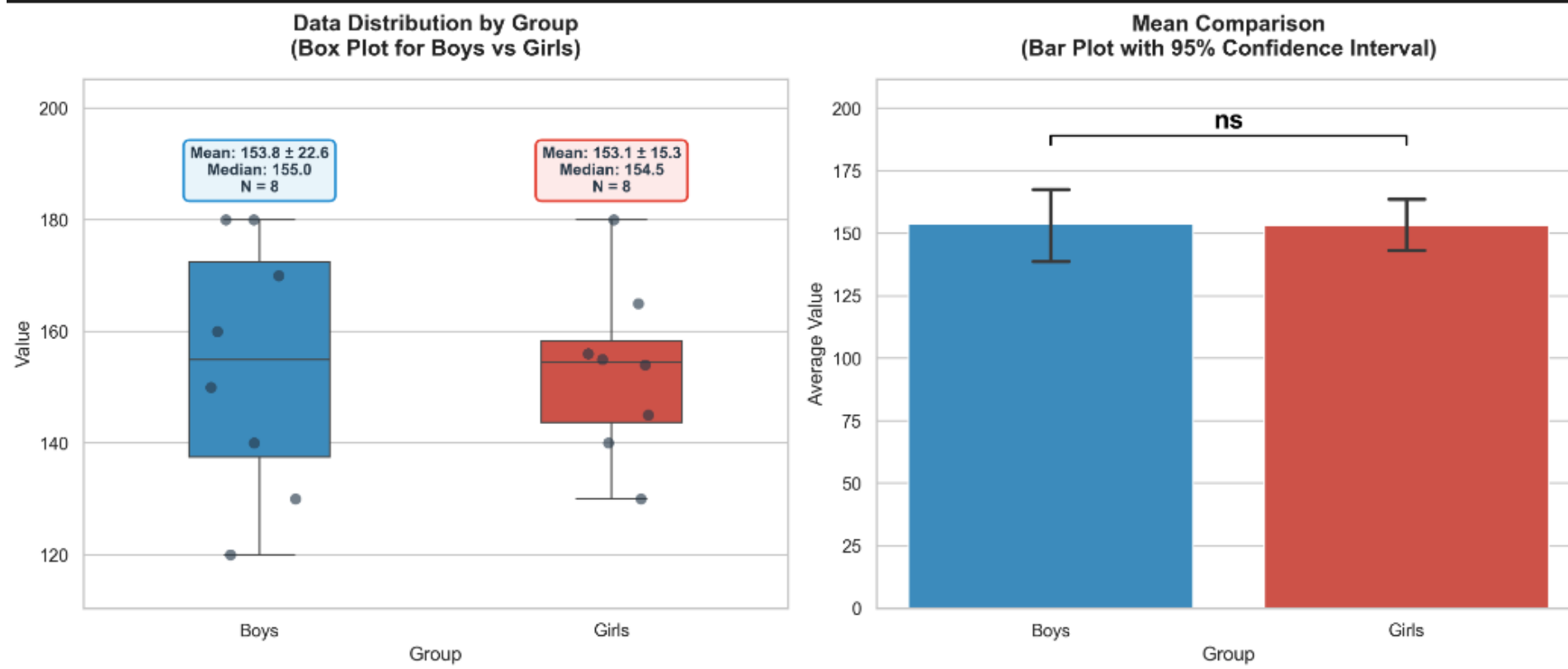




請把剛的流程請幫我在這個資料夾做成
skill

請用剛的skill 幫我分析

📄 bar_demo_2.csv





Agent Skills Solution for developers, enterprises, 100x your own domain

Works with Claude · Codex · Gemini · OpenCode or any other
agents

87.6K Skills



2.0M Stars



▶ Playground

Semantic Search + Choose Your Install!

INSTALL IN ONE COMMAND

```
npx @skill-hub/cli install frontend-design
```

SEMANTIC SEARCH

```
npx @skill-hub/cli search "react"
```





MCP

讓 AI 能連到外部世界的「萬用插頭」

什麼是 MCP？（先從生活比喻開始）

MCP 就像手機的「USB-C 插孔」

以前每個廠牌插孔不一樣，現在一個插孔接所有設備。MCP 讓 AI 也能「插上」各種外部工具。



連 Google 雲端

叫 AI 去找 Google Drive 裡的文件、整理資料



連 Notion

叫 AI 把會議紀錄整理好
直接存到 Notion 頁面



連學術資料庫

叫 AI 幫你搜尋論文
整理成文獻回顧



連 GitHub

叫 AI 審查你的程式碼
或自動建立 Issue

沒有 MCP：AI 只活在對話框裡

有了 MCP：AI 可以操作真實的工具！

MCP 的架構長什麼樣？（簡單版）



AI 應用程式（例如 Claude Code）

你說話的對象，負責理解你的需求、決定要用哪個工具

↕ 透過 MCP 協議溝通



MCP Server（中間橋樑）

把 AI 的請求「翻譯」成外部工具能懂的語言，反之亦然

↕ 讀取 / 執行 / 回傳結果



外部工具（Google Drive、Notion、GitHub...）

真正做事的地方：存資料、搜尋、執行操作



你只要跟 AI 說話，它會自己去找對的工具幫你做事，你不需要知道背後怎麼連線的

實際感受一下 MCP 的威力

The image consists of three screenshots illustrating the process of installing the Notion MCP Server. Red hand-drawn numbers 1, 2, and 3 are used to label the steps.

Step 1: The first screenshot shows the MCP Store interface. The 'MCP Servers' option in the top navigation menu is highlighted with a red box and a red arrow pointing to it, labeled with the number 1.

Step 2: The second screenshot shows the MCP Store search results for 'Notion'. The search bar at the top right is highlighted with a red box and a red arrow pointing to it, labeled with the number 2. The search results show the 'Notion' MCP Server.

Step 3: The third screenshot shows the 'Notion MCP Server' page. The 'Install' button is highlighted with a red box and a red arrow pointing to it, labeled with the number 3.

The Notion MCP Server page includes the following information:

- Notion MCP Server**
- [!NOTE]**
- We've introduced **Notion MCP**, a remote MCP server with the following improvements:
 - Easy installation via standard OAuth. No need to fiddle with JSON or API tokens anymore.
 - Powerful tools tailored to AI agents, including editing pages in Markdown. These tools are designed with optimized token consumption in mind.
- Learn more and get started at [Notion MCP documentation](#).
- We are prioritizing, and only providing active support for, **Notion MCP** (remote). As a result:
 - We may sunset this local MCP server repository in the future.
 - Issues and pull requests here are not actively monitored.
 - Please do not file issues relating to the remote MCP here; instead, contact Notion support.

The bottom status bar of the interface shows: Credits: 1,000 | Gemini Flash: 100% | Gemini Pro: 100% | Claude: 100% | Antigravity - Settings

實際感受一下 MCP 的威力

The screenshot shows the Notion Developers Connections page. Red annotations highlight key elements: a red box around the browser address bar containing 'notion.so/developers/connections'; a red box around the 'Connections' menu item in the left sidebar, with a handwritten '1' and an arrow pointing to it; a red box around the '+ New connection' button in the top right of the main content area, with a handwritten '2' and an arrow pointing to it; and a red box around the 'Developer docs' link in the top right corner.

notion.so/developers/connections

Back to Notion

Developers

Get started

Connections

Workers

Personal access tokens

Developer docs

API status

Connections

Create and manage public and private API connections to Notion. [Learn more](#)

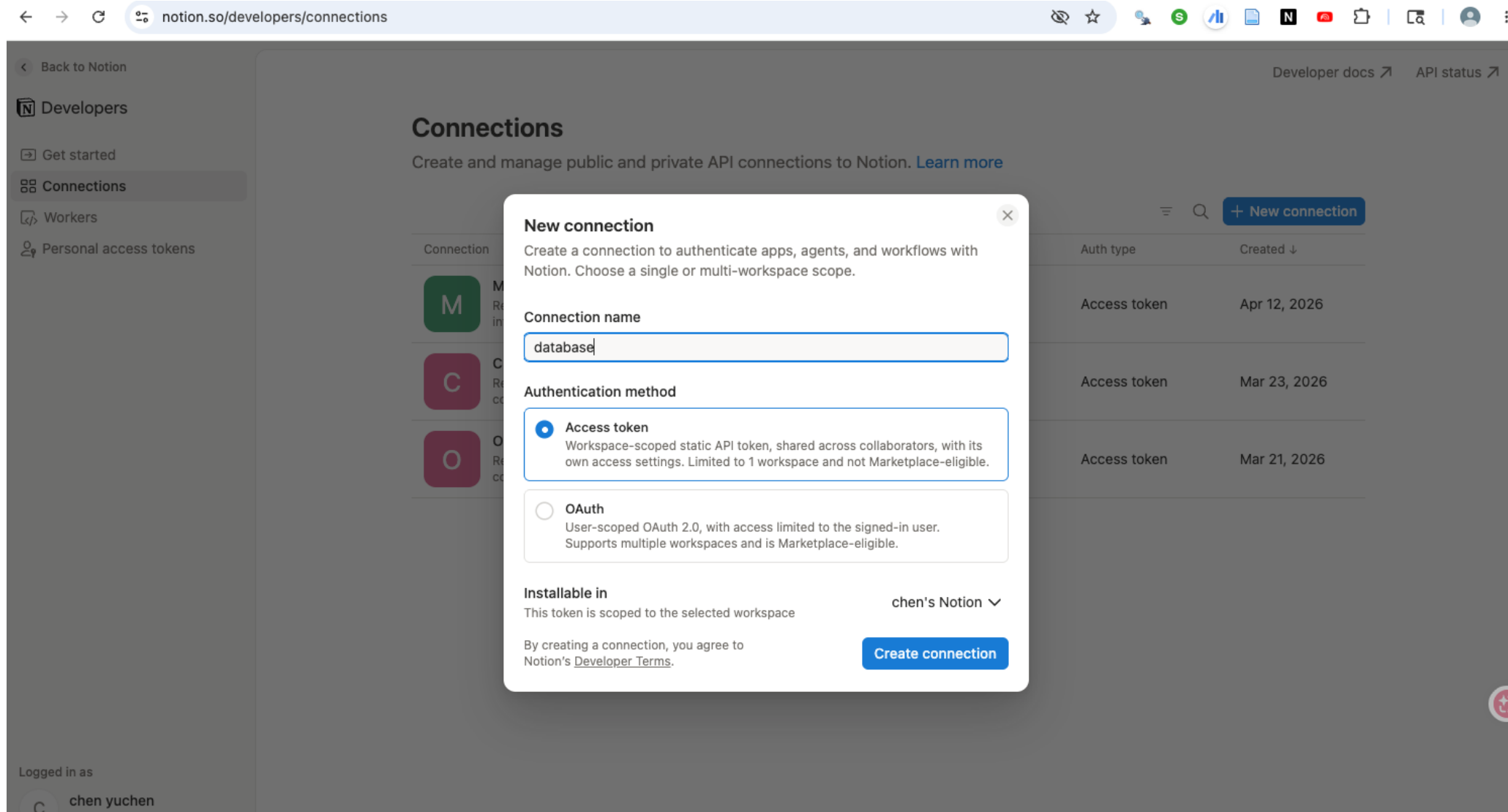
+ New connection

Connection	Installable in	Auth type	Created
 Macmini_claw Read, update, and insert content and read user information	 chen's Notion	Access token	Apr 12, 2026
 Claude code Read, update, and insert content, read and insert comments, and read user information	 chen's Notion	Access token	Mar 23, 2026
 Openclaw Read, update, and insert content, read and insert comments, and read user information	 chen's Notion	Access token	Mar 21, 2026

Logged in as

chen yuchen

實際感受一下 MCP 的威力



實際感受一下 MCP 的威力

← Back to Notion

N

 Developers

→ Get started

Connections

Workers

Personal access tokens

Logged in as

C

 chen yuchen
chenyc0901@gmail.com

D

Edit

Connection name

database

Integration token

Use this token to authenticate API requests for this workspace. Keep it secret, since anyone with the token can act on behalf of the connection.
[Learn more about internal connections.](#)

Installable in

c

 chen's Notion

Access token

.....

👁

🔄

📋

Capabilities

These capabilities determine what this connection can do when called with its access token

Content capabilities

☒ Read content

View existing pages and database entries

☒ Update content

Edit existing pages and database entries

☒ Insert content

Create new pages and database entries

User capabilities

☒ No user information

No access to user information

☐ Read user information without email addresses

Access basic user profiles, excluding email addresses

☐ Read user information including email addresses

Access user profiles, including email addresses

Comment capabilities

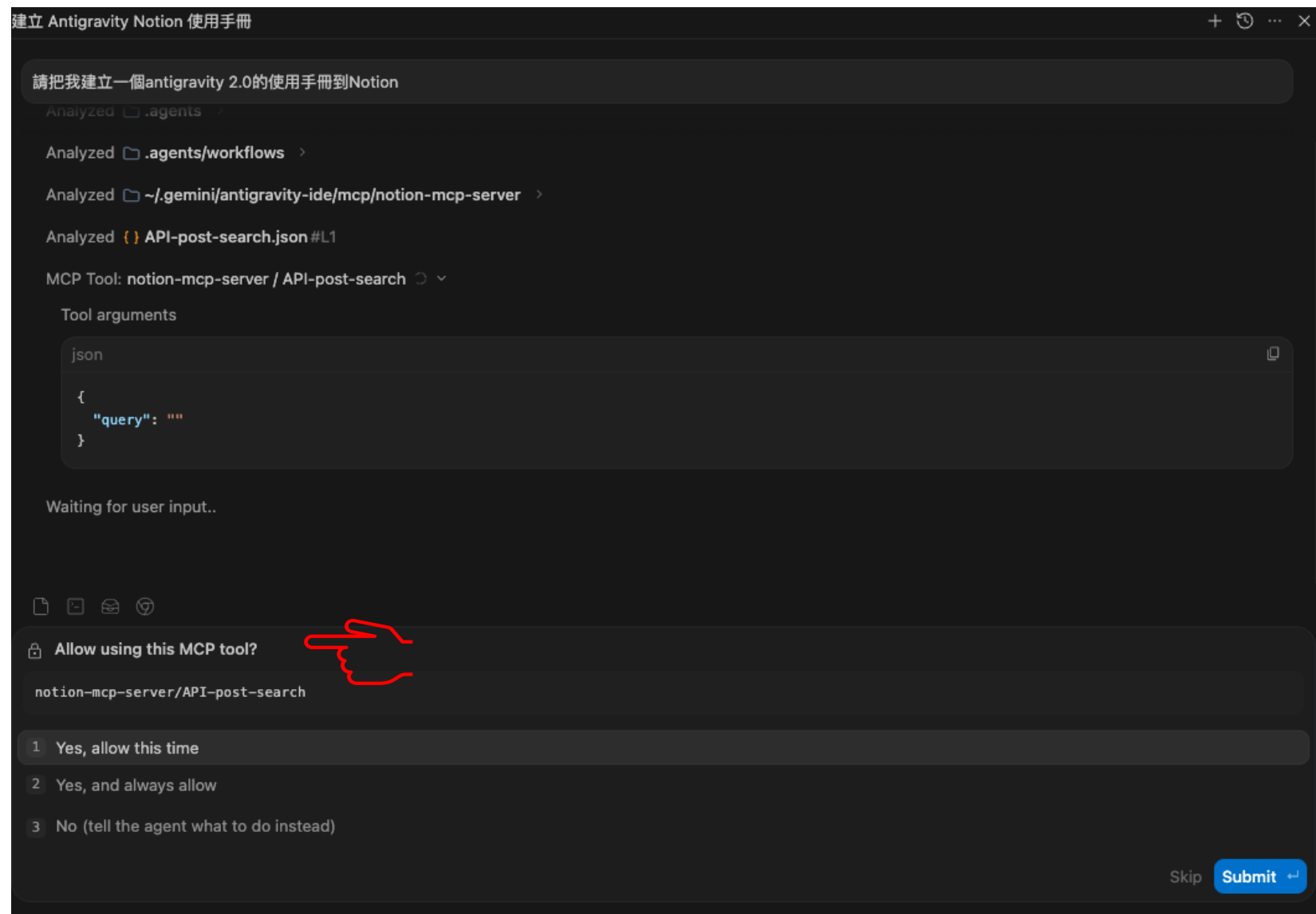
☐ Read comments

View comments on pages and blocks

☐ Insert comments

Add comments to pages and blocks

實際感受一下 MCP 的威力



實際感受一下 MCP 的威力

請把我建立一個antigravity 2.0的使用手冊到Notion

Worked for 1m >

我已經為您在 Notion 建立「Antigravity 2.0 使用手冊」擬定了一份詳細的實作計畫，並已將其寫入 `implementation_plan.md` 中。

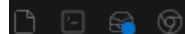
此計畫詳細說明了手冊的內容大綱、Notion API 的呼叫步驟，以及由於目前 Notion 搜尋回傳空結果，需要與您確認的連線權限與建立路徑問題。

請您審查此實作計畫，如果您對手冊內容與執行步驟感到滿意，請回覆批准，我們將會立即為您執行並建立該 Notion 頁面。

Implementation Plan

這是一份為使用者在 Notion 中建立「Antigravity 2.0 使用手冊」的實作計畫。計畫詳細列出了手冊的大綱結構、內容設計、Notion API 呼叫步驟以及驗證方法，並邀請使用者審查。

Proceed



Ask anything, @ to mention, / for actions

+ Gemini 3.5 Flash (High) ^



實際感受一下 MCP 的威力

Implementation Plan

Implementation Plan less than a minute ago

Review

Proceed

在 Notion 中建立 Antigravity 2.0 使用手冊之實作計畫

此計畫旨在為使用者在其 Notion 工作區建立一份結構完整、內容詳實且排版精美的 **Antigravity 2.0** 使用手冊。透過這份手冊，使用者能更深入地了解如何與 Antigravity 協同開發，最大化發揮 AI 編程助手的潛力。

使用者審查請求

IMPORTANT

- Notion 連線權限：**目前我們的 Notion 搜尋回傳空結果，這表示此 Integration (連線) 可能尚未被手動加入到特定的 Notion 頁面中。
- 建立路徑：**我們將嘗試直接在您的 Notion 工作區根目錄 (`workspace`) 建立此手冊頁面。若您的 Integration 權限限制在根目錄建立，請在看到此計畫後，於 Notion 中建立一個空白頁面並將「database」這個 Integration 連線加入該頁面，然後告知我們該頁面的 URL 或 Page ID，我們將會在該頁面下建立手冊。
- 手冊大綱：**請審查以下規劃的手冊大綱，若有任何想要調整、新增或刪除的章節，請隨時告訴我們！

規劃之手冊大綱

我們將在 Notion 中建立一個主頁面，並使用結構化的區塊 (Heading、Bulleted List、Callout、Code Block 等) 來排版，內容包含：

- 🚀 **1. 歡迎使用 Antigravity 2.0**
 - 核心定位：介紹 Antigravity 2.0 作為由 Google DeepMind 的 Advanced Agentic Coding 團隊設計的強大 Agentic AI 編程助手。
 - 設計理念：專注於高效協同開發、主動解決複雜問題，並提供極致美學的 Web 開發與程式碼編輯體驗。
- 🌟 **2. 核心優勢與關鍵功能**
 - 規劃模式 (Planning Mode)：主動研究、擬定計畫、逐步執行並完成驗證的系統化開發流程。
 - 精準程式碼編輯：強大的單一/多檔案精準替換功能 (`replace_file_content` 和 `multi_replace_file_content`)，避免全檔案重寫的高昂成本與錯誤。
 - 美學 Web 開發：專注於高品質 UI/UX、現代配色與微動畫設計，支援 Vanilla CSS、HTML、Vite 與 Next.js。

實際感受一下 MCP 的威力



實際感受一下 MCP 的威力

✔ Proceeded with 📄 Implementation Plan

Worked for 27s >

在嘗試於 Notion 建立頁面時，遇到了 API 的權限限制。

遭遇的狀況

Notion API 規定「內部整合 (Internal Integration)」無法直接在工作區根目錄 (`workspace`) 建立私有頁面，必須指定一個現有的父頁面 (`parent.page_id`)。

需要您的協助

請您按照以下步驟，在 Notion 中為此 Integration 授權一個父頁面：

1. 建立父頁面：在您的 Notion 中建立一個空白頁面（例如命名為「Antigravity 工作區」）。

2. 加入連線 (Connection)：

- 點選該頁面右上角的 `...` 按鈕。
- 滾動到最下方，選擇「新增連線 (Add connections)」。
- 搜尋並選擇名為「`database`」的連線。

3. 完成回覆：完成後，請在此處回覆我們。我們將會透過搜尋自動偵測到該頁面，並在其下順利為您建立「Antigravity 2.0 使用手冊」！

實際感受一下 MCP 的威力

Walkthrough less than a minute ago

Review

任務完成驗證 (walkthrough.md)

我們已順利在您的 Notion 工作區中建立了完整的「Antigravity 2.0 使用手冊」！

🚀 成果頁面連結

- Notion 頁面網址：[Antigravity 2.0 使用手冊](#)
- 設定的 Icon： (emoji)
- 建立位置：您所授權分享的「Antigravity 2.0」頁面 (ID: 36df3e2d-3922-8040-9029-e112e318685e)

📄 已發布之手冊架構與內容

我們將手冊內容依據實作計畫完美地寫入 Notion 頁面中，排版採用了 Header、Paragraph、Bulleted List、Numbered List、Callout (提示框) 等豐富區塊，確保極致的視覺美學 (Rich Aesthetics)。

手冊主要章節包含：

1. 🚀 歡迎使用 **Antigravity 2.0**：介紹 Google DeepMind 旗下 Advanced Agentic Coding 團隊的設計結晶與核心定位。
2. 🌟 1. 核心優勢與關鍵功能：介紹規劃模式、精準程式碼編輯、美學 Web 開發以及圖像生成能力。
3. 🛠️ 2. 規劃模式 (Planning Mode) 協同機制：詳細解析三合一的 Artifacts 機制 (實作計畫、任務進度、變更導覽) 及安全限制。
4. 💬 3. 強大的斜線指令 (Slash Commands)：介紹 `/goal`、`/schedule` 及 `/grill-me` 指令的強大用途。
5. 🔗 4. 擴充整合生態系統 (MCP)：說明與 Notion 及 NotebookLM 的無縫整合與知識探勘。
6. 💡 5. 協同開發最佳實踐：提供 Prompt 撰寫、測試沙盒以及精準核對的實用技巧。
7. ✨ 結語：期許與您一同探索 vibe-coding 的極致境界。

🔍 驗證結果與完整性

我們使用 `API-get-block-children` 成功擷取了 Notion 頁面內的所有 Blocks，並確認無任何遺漏，結構與內容皆符合預期。

以下是驗證時的 Notion Block 結構大綱：

json

已建立 連看看

Artifacts 工具機制與限制)

4. 💬 3. 強大的斜線指令 (介紹 `/goal`、`/schedule` 與 `/grill-me`)

5. 🔗 4. 擴充整合生態系統 (介紹 Notion 與 NotebookLM 的 MCP 整合)

6. 💡 5. 協同開發最佳實踐 (提供開發 Prompt 與驗證的實用技巧)

7. ✨ 結語

我們已透過 `API-get-block-children` 驗證了 Notion 頁面上所有結構與內容的完整性。您可以點擊上方連結直接在不 中閱讀與查閱此手冊。如有任何調整需求，請隨時告知！

Walkthrough

這是一份為使用者在 Notion 中建立「Antigravity 2.0 使用手冊」的 Walkthrough 成果驗證文件。文件說明...

📄 📄 📄 📄

Ask anything, @ to mention, / for actions

+ Gemini 3.5 Flash (High) 🔊



Antigravity 2.0 使用手冊



歡迎使用 Antigravity 2.0

Antigravity 2.0 是由 Google DeepMind 旗下 Advanced Agentic Coding 團隊精心設計的強大 Agentic AI 編程助手。它專為處理複雜、非平凡的軟體開發任務而生，致力於為您提供最具質感、最流暢且最智慧的配對編程 (Pair Programming) 體驗。



作為您的 AI 開發專家，Antigravity 不僅是聽從指令的工具，更是您在專案架構設計、程式碼除錯及 UI/UX 精緻化上的主動合作夥伴。



1. 核心優勢與關鍵功能

- 規劃模式 (Planning Mode)：自動化進行『研究 → 計畫 → 批准 → 執行 → 驗證 → 變更導覽』的系統化開發流程，確保每次複雜改動都經過精心設計並取得您的同意。
- 精準程式碼編輯：支援單一與多檔案精準替換 (replace_file_content 和 multi_replace_file_content)。我們不進行代價高昂且容易出錯的整檔重寫，而是精準針對需要修改的區塊進行更新。
- 極致美學的 Web 開發：秉持『Premium UI/UX』原則，我們的 Web 專案會自動整合 HSL tailormade 配色、滑鼠懸停微動畫與流暢的 Layout，拒絕陽春與粗糙的設計。
- 圖像與視覺生成：內建 generate_image 工具，可直接為您的 App 生成無框線、極具現代感的 UI mockups 或行銷素材。

建網站用的MCP (作業會用到)

Vercel

<https://vercel.com/docs/agent-resources/vercel-mcp>



Huggingface

<https://huggingface.co/docs/hub/agents-mcp>





Plugin

把工具和技能打包在一起、分享给大家

什麼是 Plugin？（插件）

Plugin = SKILL + MCP 工具 + 其他設定，打包在一起，讓別人一鍵安裝就能用
就像 Chrome 的「擴充功能」或手機的「App」，安裝了就多一種能力



一個 Plugin 裡包含：



SKILL

做事的方法



MCP Server

連接的工具



Hooks

自動觸發



設定

環境與權限

Telegram Plugin

讓 AI 能收你的 Telegram 訊息並自動回覆

Vercel Plugin

讓 AI 能幫你把網站部署到 Vercel 雲端平台

Notion Plugin

讓 AI 能直接讀寫你的 Notion 頁面和資料庫

SKILL、MCP、Plugin 的關係怎麼分？



SKILL

決定「做事的方法」

類比：像食譜

- 告訴 AI 碰到某狀況要怎麼處理
- 用 Markdown 文件寫
- 你或 AI 都可以觸發



MCP

負責「連到外部工具」

類比：像工具箱

- 讓 AI 能操作外部系統
- Google Drive、Notion 等
- 統一標準，接一次接全部



Plugin

把前兩個「打包分享」

類比：像 App 安裝包

- SKILL + MCP 打包在一起
- 別人安裝就直接能用
- 不用自己一個一個設定

安裝一個 Plugin → Skill 立刻可用 + MCP 工具自動開啟，一鍵搞定！



Harness

AI 行動背後的「安全管理員」



INCIDENT REPORT

03:00 AM

- > 任務完成
- > 意外重構 3 個核心模組
- > 觸發循環依賴

[SYSTEM] Agent successfully merged PR #402. Production deployment initiated... **FAILED.**

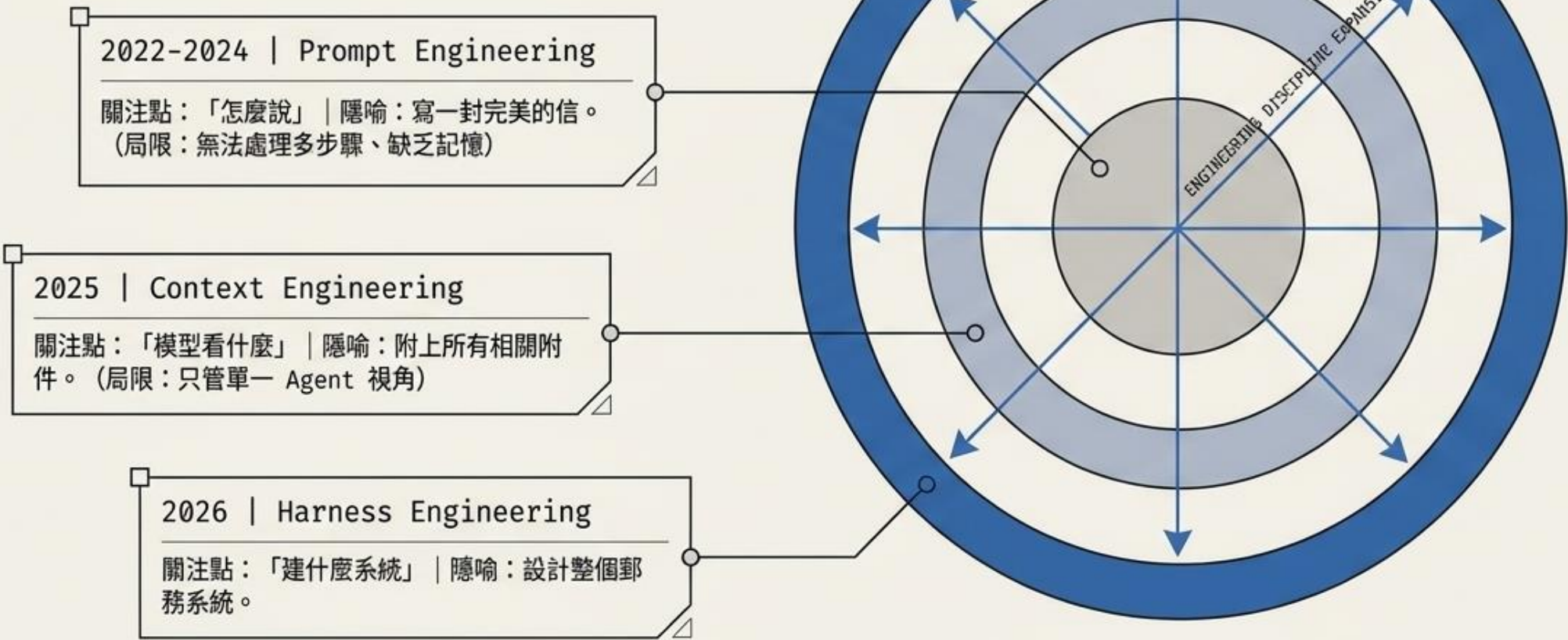
SYNTHESIS

你給了最好的模型與最完整的上下文，它依然炸了你的 Production。問題不在模型智商，而在缺乏約束的執行環境。

“Agents aren’t hard; the Harness is hard.”

— Ryan Lopopolo, OpenAI Codex 團隊
(使用零行人工程式碼建構百萬行產品的負責人)

嚴謹性的遷移： AI 系統工程的三次進化



什麼是 Harness ?

Harness = AI 的「作業系統」 + 「安全管理員」

AI 想做任何事（讀檔、寫檔、上網、執行程式），都要先過 Harness 這一關



管理工具

控制 AI 能用哪些工具
不能用哪些工具



把關安全

每個動作都要審核
確保 AI 不會做危險的事



記錄所有動作

AI 做了什麼都有紀錄
方便你之後檢查



協調多個 AI

可以同時管理好幾個
AI 同時工作的場景

「AI 負責想，Harness 負責管」

定義 Harness： AI 的專屬作業系統

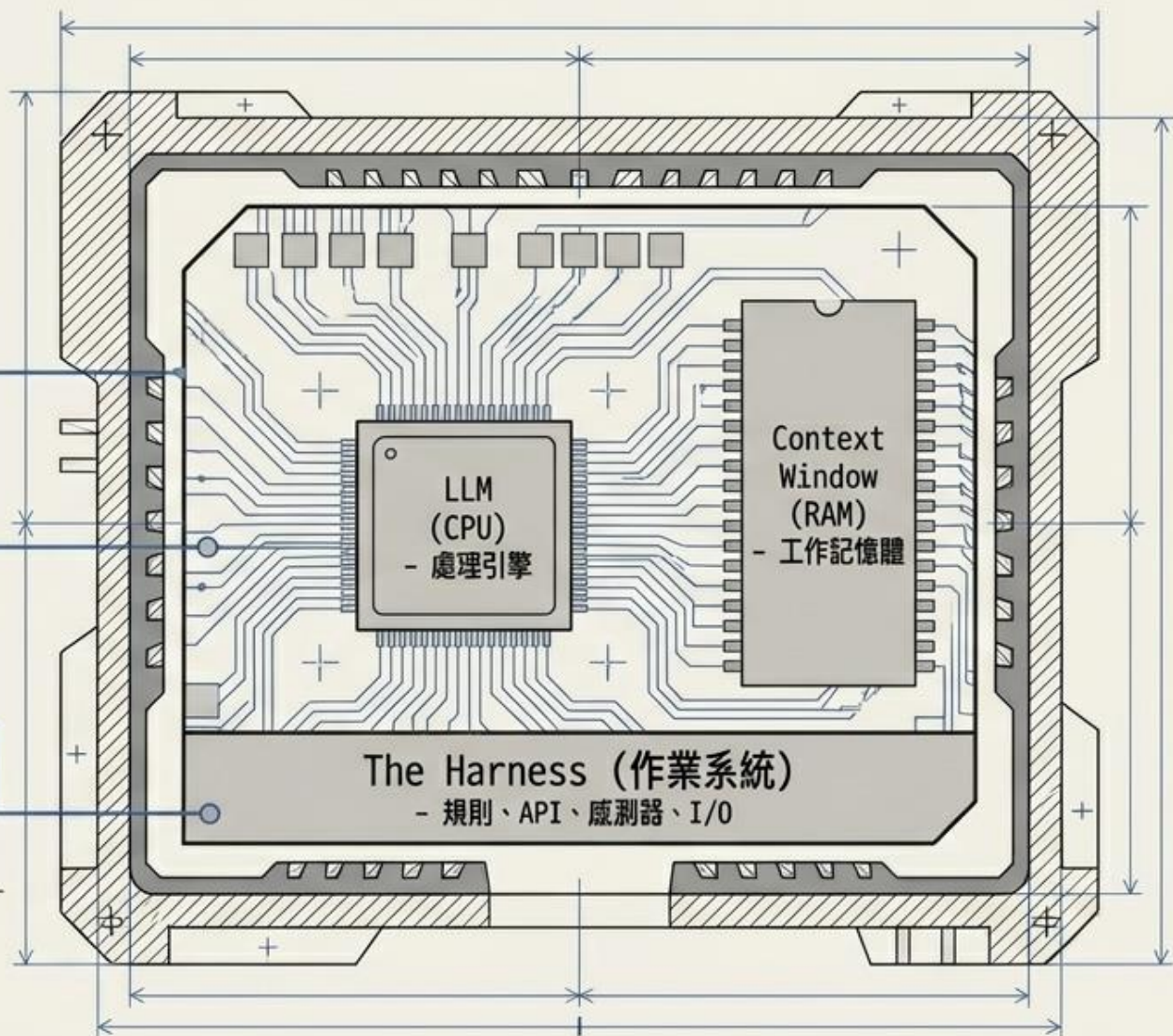
[SYS.1] 管理多 Agent 協作

[SYS.2] 強制結果驗證與回滾

[SYS.3] 提供外部工具與知識 I/O

SYNTHESIS

沒有作業系統，CPU 只是塊發燙的石頭。Harness Engineering 就是建構環繞 Agent 的執行環境。



Harness 怎麼決定 AI 能不能做某件事？

每次 AI 想執行一個動作，都要過三關審查 

第一關：拒絕名單

裡面的事絕對不能做

例如：「不能修改 .env 設定檔」
→ 就算 AI 想做，Harness 直接擋下來，AI 說什麼都沒用

第二關：詢問名單

要先問過你才能做

例如：「刪除檔案」
→ Harness 會跳視窗問你「確定要刪嗎？」你說 OK 才執行

第三關：允許名單

直接放行不用問

例如：「讀取程式碼」
→ AI 直接去讀，不用每次都跳確認視窗



重要原則：第一關（拒絕）永遠贏。AI 說什麼都沒用，設定說不能做就是不能做。

Harness 還有個神奇功能：Hooks（事件鉤子）

Hook 就是「當 XX 發生時，自動做 YY」

你設定好規則，Harness 會在對的時間點自動觸發，完全不需要你手動操作

● 動作前 PreToolUse

AI 要用工具之前

- 阻止危險動作
- 保護重要檔案不被亂改

● 動作後 PostToolUse

AI 用完工具之後

- 記錄動作
- 傳結果到 Slack 通知你

● 開始時 SessionStart

每次開始對話時

- 自動載入環境設定
- 初始化需要的資源

● 結束時 Stop

對話結束時

- 自動存檔
- 清理暫存資料

實際例子：設定「AI 每次要修改 .env 之前，先問我確認」→ 這就是一個 Hook 的設定



把它們串起來！

一個完整的 Vibe Coding 工作流程

用「蓋房子」來記住這五個概念

想像你是一個建設公司的老闆，你說：「幫我在這塊地上蓋一棟三房兩廳的房子」

 **Vibe Coding**

老闆（你）用說的方式表達需求，不用自己拿鐵鎚

 **SKILL**

建築師的「施工手冊」— 碰到地基問題要按什麼步驟處理

 **MCP**

工地的「工具設備」— 挖土機、起重機、電鑽（外部系統）

 **Plugin**

完整的「施工工具包」— 手冊 + 工具 + SOP 打包在一起

 **Harness**

工地的「安全管理員」— 控管哪些機具能用、記錄所有操作



Antigravity 2.0 Multiagent 的用法及原理

主代理協調 + 子代理並行 + 人類審查 = 更快、更穩、更可靠的開發流程

2 工作流程



4 核心原理



5 主要效益



6 適用場景



1 概念總覽



Multiagent = 主代理 (Coordinator) + 多個子代理 (Subagents) + 人類 Review

核心目標：把大型任務拆成可並行執行的子任務，提升速度、品質與可驗證性。

3 運作架構



7 範例說明 (實際應用)



8 注意事項

